Product Management System Using OOP and Exception Handling

Problem Statement

Consider an E-Commerce Product Management System in which a user can create a product, update its price, and display its details. The system must handle exceptional situations, such as entering a negative product price or attempting to update a product with an invalid price.

Develop a Java application using Object-Oriented Programming (OOP) concepts to demonstrate the use of `try`, `catch`, and `finally` blocks.

Application Requirements

1. Create a class named `Product` with the following private data members:

- `productId` (int)
- `productName` (String)
- `category` (String)
- `price` (double)

2. Implement a parameterized constructor to initialize the product details.

3. Define the following methods:

- `updatePrice(double newPrice)`: Updates the product price after validating the new price.
- `displayProduct()`: Displays all product details.
- `getPrice()`: Returns the current product price.

4. Use a `try` block to perform product creation and price update operations that may generate exceptions.

5. Use appropriate `catch` blocks to handle:

- `IllegalArgumentException` when the product price is zero or negative.
- `NumberFormatException` when a price entered as text cannot be converted into a numeric value.

6. Use a `finally` block to display a message indicating that the product operation has been completed, regardless of whether an exception occurs.

7. In the `main()` method, create a `Product` object, attempt to update its price, handle exceptions, and display the final product details.

Expected Behaviour

Suppose a product has the following details:

- Product ID: 101
- Product Name: Laptop
- Category: Electronics
- Price: ₹50,000

The user attempts to update its price to `-5000`.

- The `try` block attempts to update the price.
- The `catch` block handles the `IllegalArgumentException` and displays an appropriate error message.
- The `finally` block displays the operation completion message.
- The product retains its original price because the update failed.

Concepts Demonstrated

- Encapsulation: Private data members and controlled access through methods.
- Classes and Objects: Creating and managing product objects.
- Exception Handling: Using `try` and `catch` to handle invalid input.
- Finally Block: Executing a message or cleanup operation regardless of success or failure.

Learning outcome: Students will understand how OOP principles and exception handling can be combined to build a reliable product management application.